

Bioinformática

Ensamblado

Rodrigo Santamaría

Ensamblado

- El problema del ensamblado
- Grafos de De Bruijn
- Teorema de Euler
- Consideraciones adicionales

El problema del ensamblado

El problema del periódico

Ordenación, composición y reconstrucción

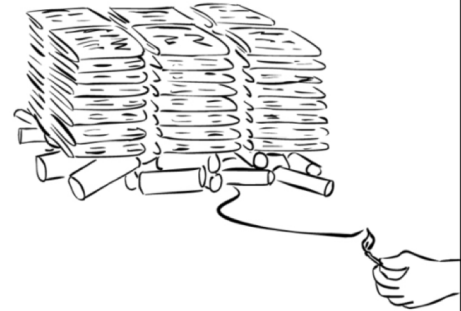
El problema del periódico



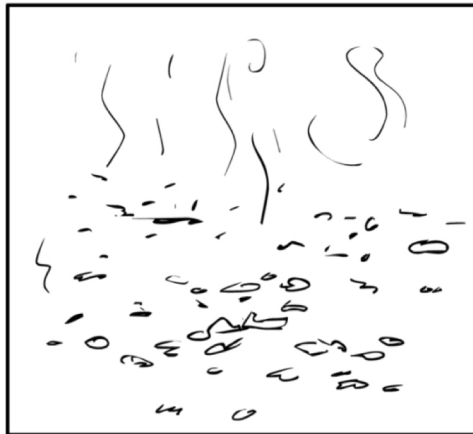
stack of NY Times, June 27, 2000



stack of NY Times, June 27, 2000
on a pile of dynamite



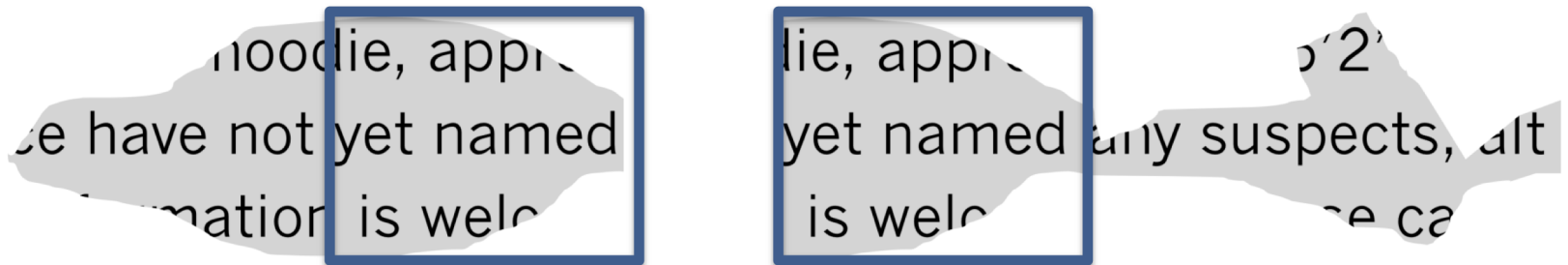
this is just hypothetical



so, what did the June 27, 2000 NY
Times say?

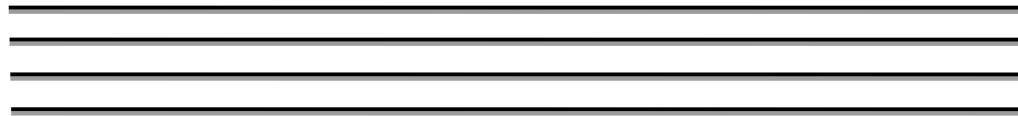
El problema del periódico

- Dificultades
 - Pérdida y descontextualización de información
 - Múltiples copias de la misma información

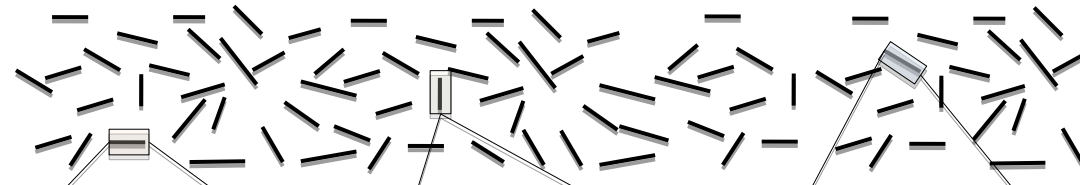


El problema del ensamblado

múltiples copias
idénticas de un genoma



rompemos el genoma
en lecturas (**reads**)*



secuenciamos los reads

AGAATATCA

TGAGAATAT

GAGAATATC

ensamblamos el
genoma haciendo uso
de reads solapados

AGAATATCA
GAGAATATC
TGAGAATAT
...TGAGAATATCA...

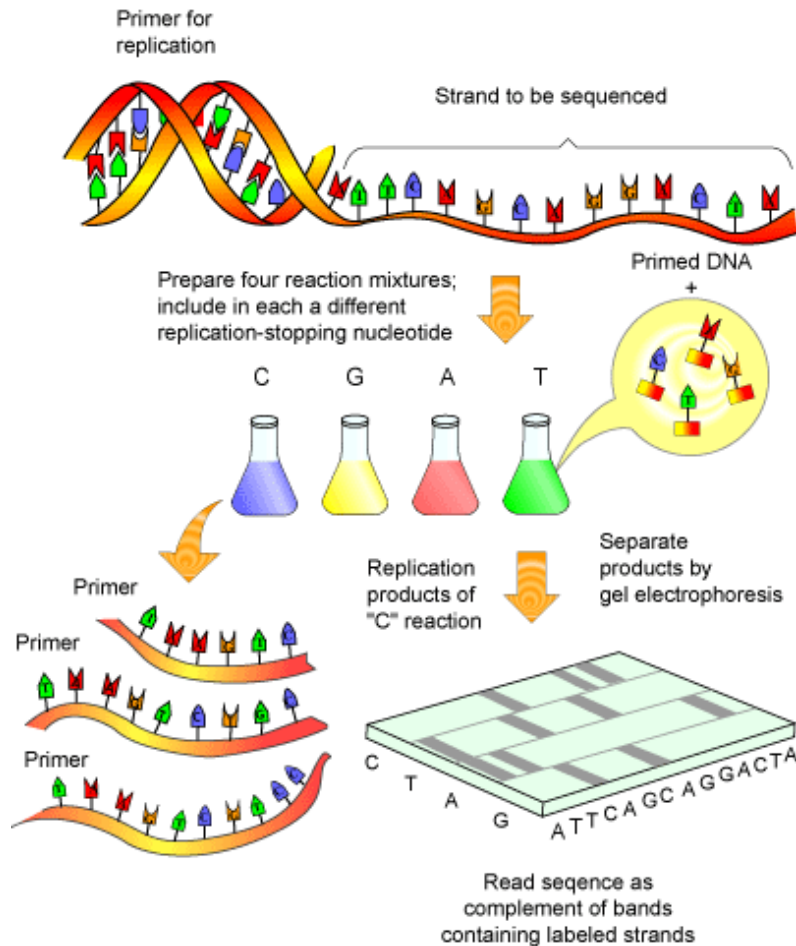
*¿por qué tenemos que romper el genoma?

¿Por qué tenemos que romper?

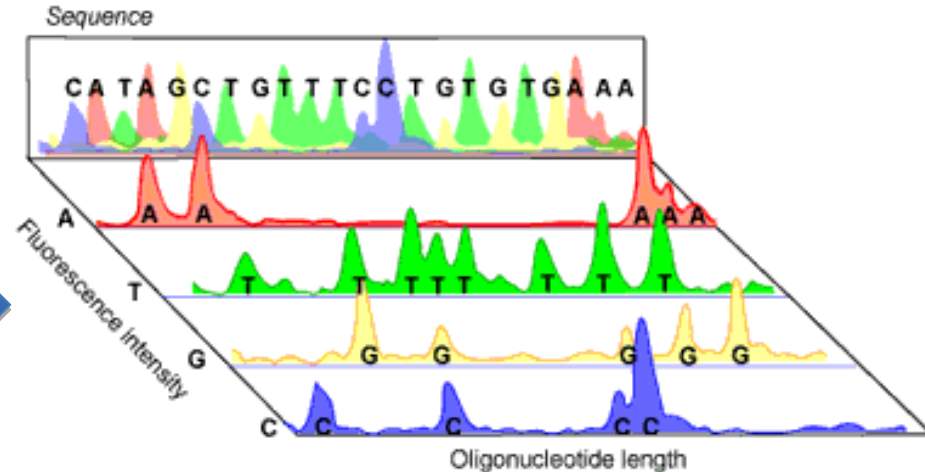
- La tecnología actual pierde precisión de lectura, o no es aplicable, en secuencias largas
- Es posible que en un futuro próximo se reduzca este problema*
 - Oxford Nanopore:
 - <http://www.ebi.ac.uk/about/news/press-releases/mini-dna-sequencer-tests-true>
 - Reads de tamaño ilimitado en la teoría, de momento unos 10-12Kbps (100-150 bps en Illumina).

*Más precisamente, se habla de tecnologías High Throughput (de alto rendimiento) de Lectura Corta (SR) y Lectura Larga (LR). La HT-SR se denomina también shotgun sequencing (secuenciación a bocajarro), mientras que las nuevas tecnología HT-LR incluyen el MiniON de Oxford Nanopore o los secuenciadores PacBio.

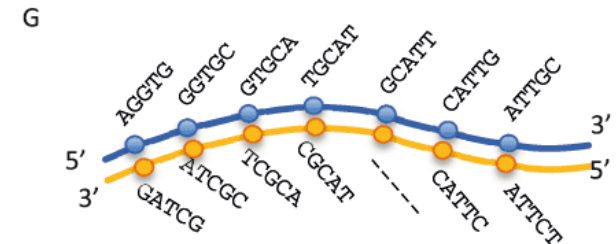
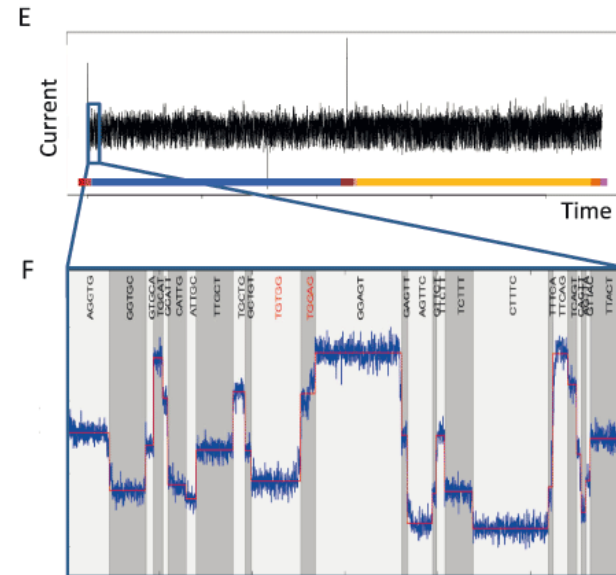
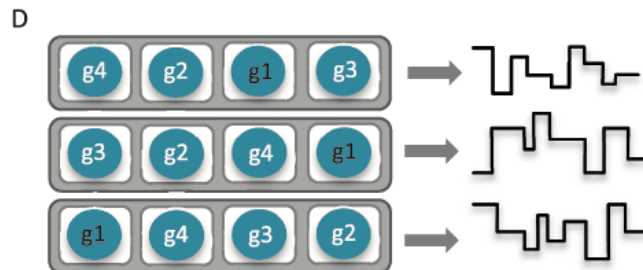
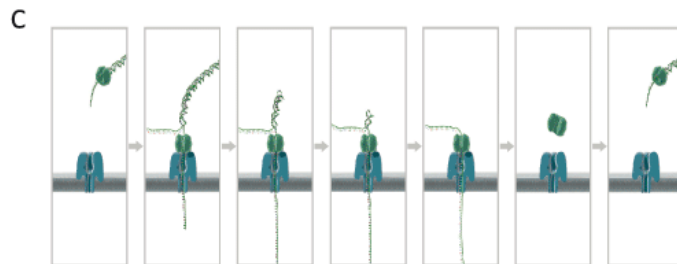
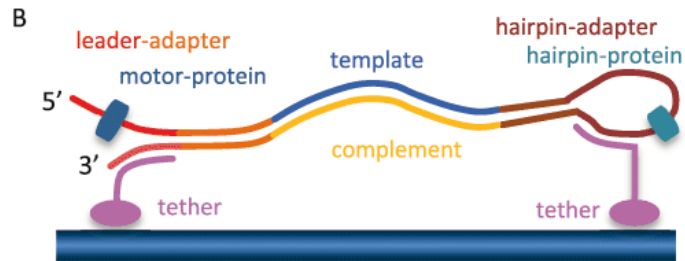
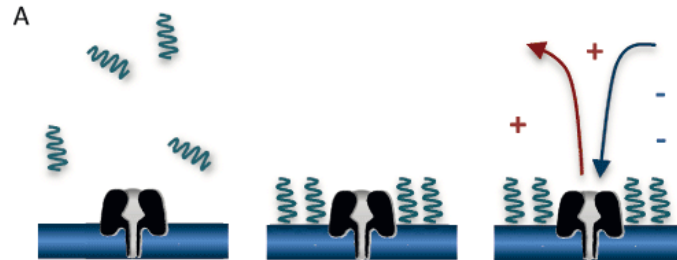
Secuenciación Sanger



No es la tecnología más actual (de hecho es la primera)
Pero nos da una idea de la complejidad*



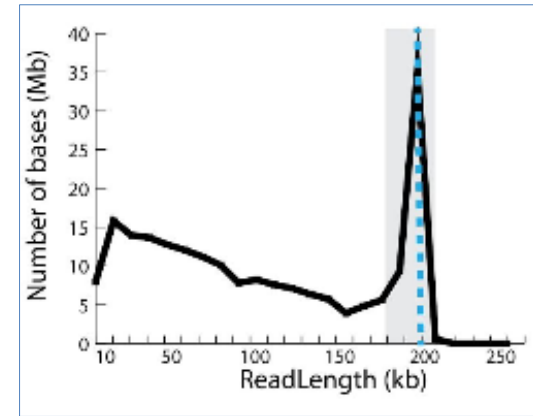
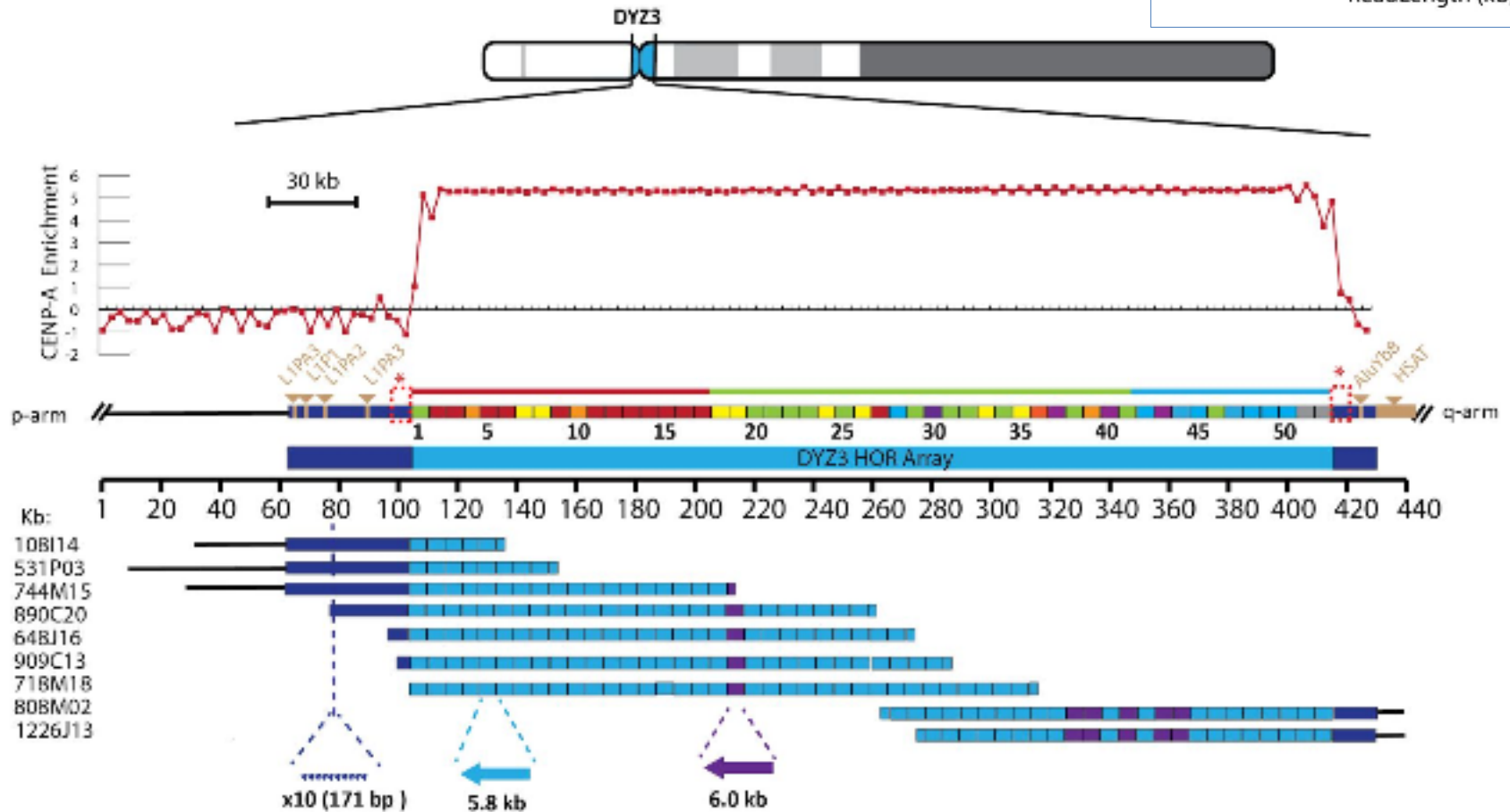
Oxford Nanopore



Oxford Nanopore

Jain et al. *Linear Assembly of a Human Y Centromere using Nanopore Long Reads*. Nature Biotechnology. **2017**

<https://www.biorxiv.org/content/early/2017/07/31/170373>



El problema del ensamblado

- Dificultades *adicionales*
 - El DNA tiene dos cadenas
 - En algunas técnicas, no sabemos a priori si una lectura nos da una cadena o su inversa complementaria
 - Las técnicas de secuenciación introducen errores
 - Evitando la identificación de todos los reads solapados
- Asunciones (iniciales)
 - Todas las lecturas tienen el mismo tamaño k
 - Todas vienen de la misma cadena y no hay errores

Ordenación

```
> a = [5, 2, 3, 1, 4]
> sorted(a)
a [1, 2, 3, 4, 5]
> a.sort()
> a
a [1, 2, 3, 4, 5]
> gente = [ ('john', 'A', 15),
             ('jane', 'B', 12),
             ('dave', 'B', 10) ]
> sorted(gente, key=lambda gente: gente[2])
[('dave', 'B', 10), ('jane', 'B', 12),
 ('john', 'A', 15)] #sorted by age
```

lambda es simplemente una forma de definir una función sin nombre en una sola línea.

Ejercicio

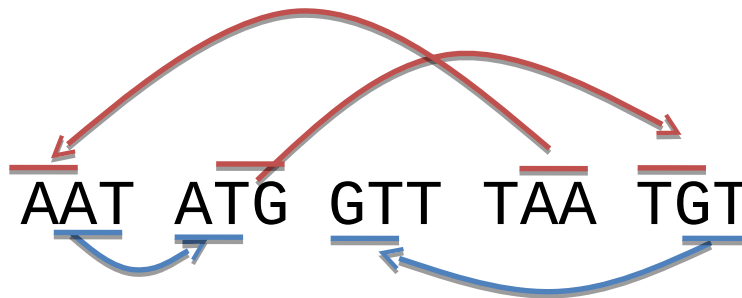
- Implementar la función `composition`:
 - **entrada**:
 - `text` (una secuencia de DNA)
 - `k` (un número entero positivo)
 - **salida**: cadena ordenada alfabéticamente de todos los posibles k-mers en `text`
- Ejemplo:
 - `text`: CAATCCAAC
 - `k`: 5
 - `salida`: ['AATCC', 'ATCCA', 'CAATC', 'CCAAC', 'TCCAA']

De composición a reconstrucción

- `composition` “imita” el proceso de secuenciación: rompe una cadena en *reads*
- Buscamos solucionar el problema inverso
 - **Entrada:** un valor k y una colección de k -mers
 - **Salida:** una cadena de texto con la reconstrucción de la cadena original a partir de los k -mers
- Este problema resulta bastante más complejo

Reconstruyendo

- La reconstrucción es sencilla si sólo hay una opción de enlace:



- El problema es cuando tenemos varias:

AAT	ATG	ATG	ATG	CAT	CCA	GAT	GCC
GGA	GGG	GTT	TAA	TGC	TGG	TGT	

Ensamblado

- El problema del ensamblado
- Grafos de De Bruijn
- Teorema de Euler
- Consideraciones adicionales

Grafos de De Bruijn

Grafos, caminos Hamiltonianos

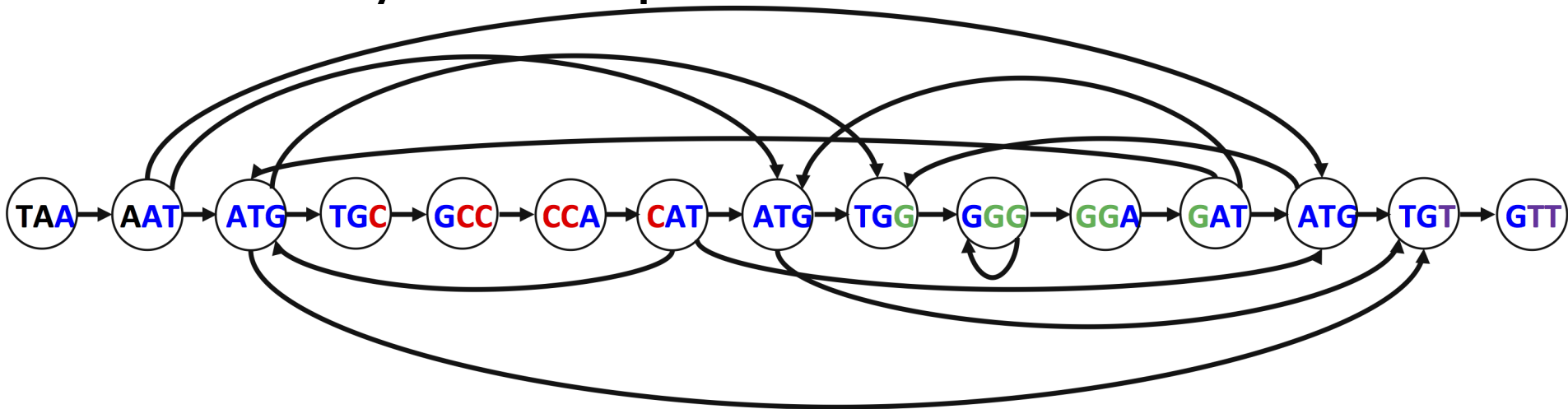
Grafos de De Bruijn, caminos Eulerianos

Grafos

- Representación de la reconstrucción

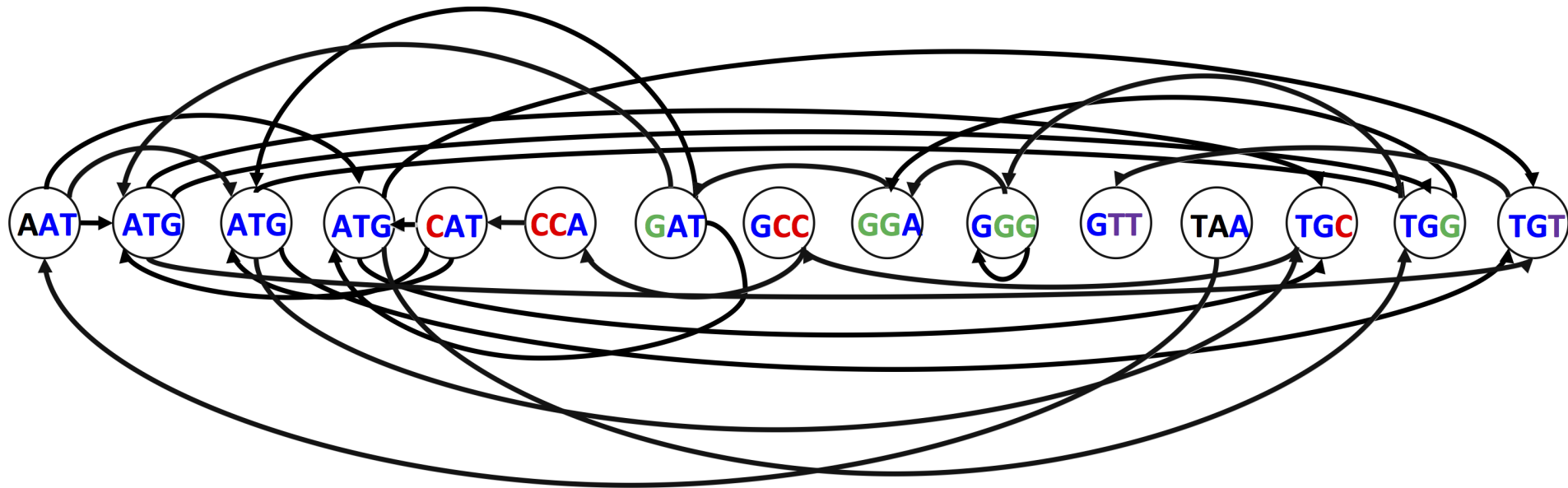


- Pero hay varias opciones de reconstrucción:



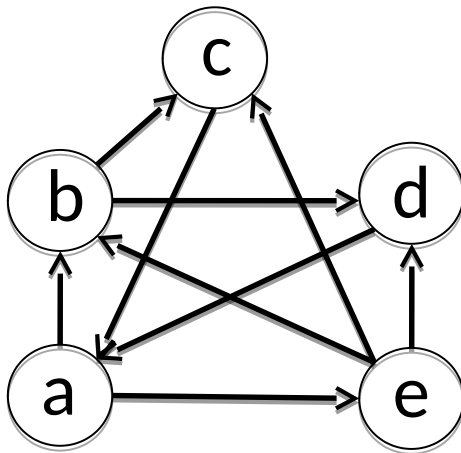
Grafos

- Y no tenemos un orden pre-establecido:



Grafos como estructura de datos

- Matrices y listas (o diccionarios) de adyacencia



	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>a</i>	0	1	0	0	1
<i>b</i>	0	0	0	1	0
<i>c</i>	1	0	0	0	0
<i>d</i>	1	0	0	0	0
<i>e</i>	0	1	1	1	0

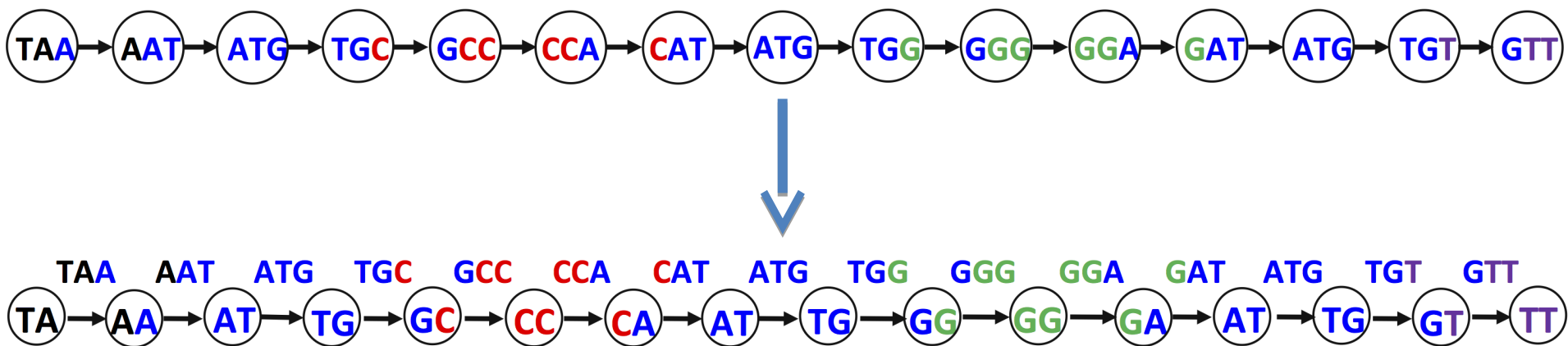
{ *a*: [b, e];
 b: [d];
 c: [a];
 d: [a];
 e: [b, c, d] }

Camino Hamiltoniano

- Ruta que visita **todos los nodos** en un grafo dirigido exactamente una vez
 - Puede haber varios caminos hamiltonianos
- Reconstruir una cadena a partir de trozos de secuencias = Encontrar un camino hamiltoniano en un grafo

Grafo de De Bruijn

- Usamos los k-mers como aristas en vez de como nodos

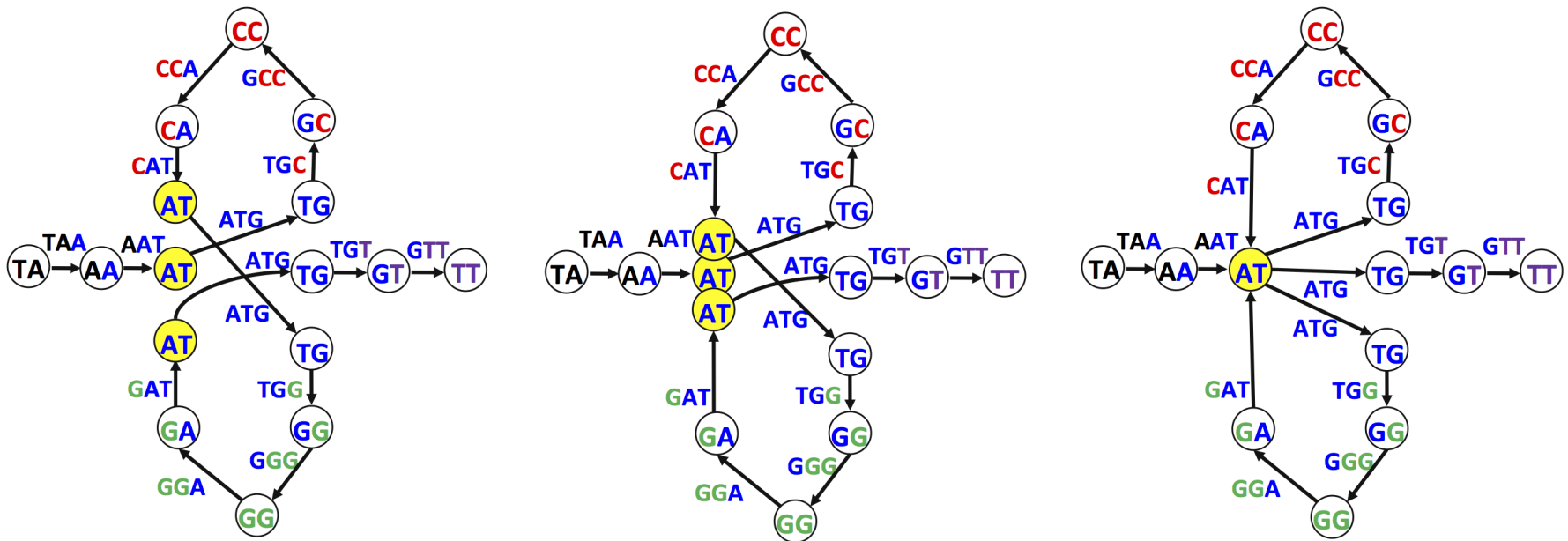


- A los nodos los etiquetaremos con los prefijos o sufijos* correspondientes a cada conexión

*Vamos a entender siempre prefijos o sufijos de una cadena S de tamaño n como $S[:n-1]$ y $S[1:]$, respectivamente

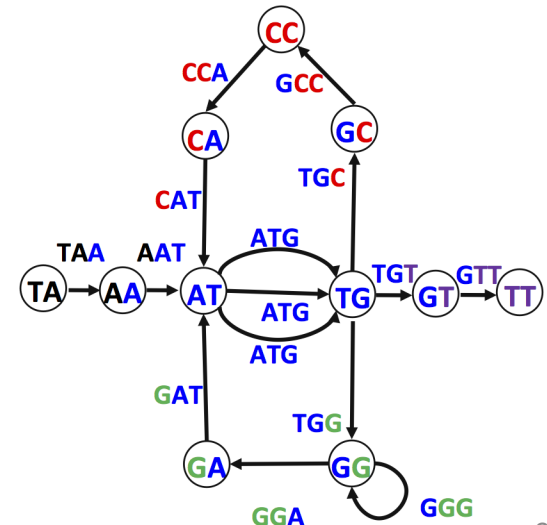
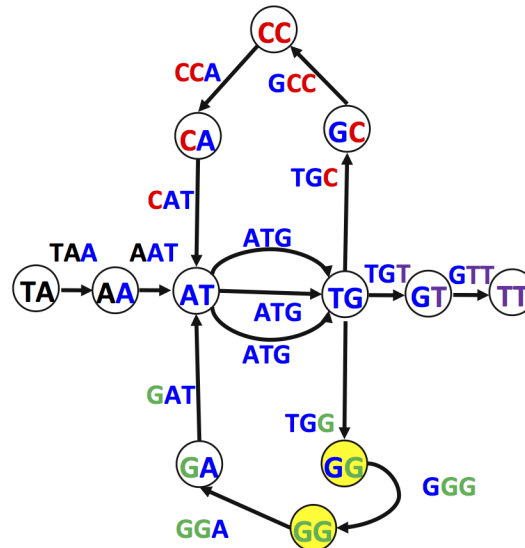
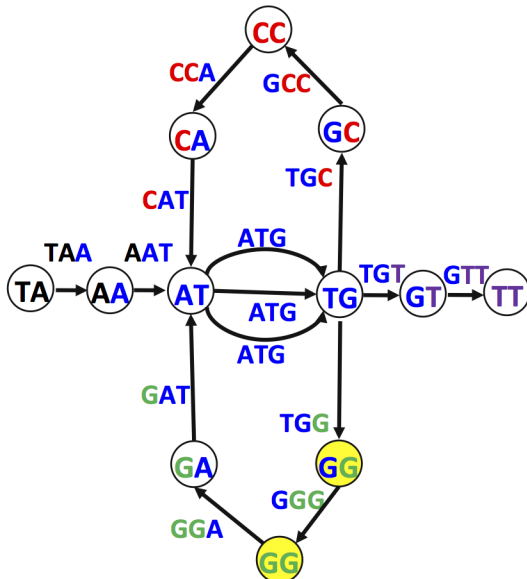
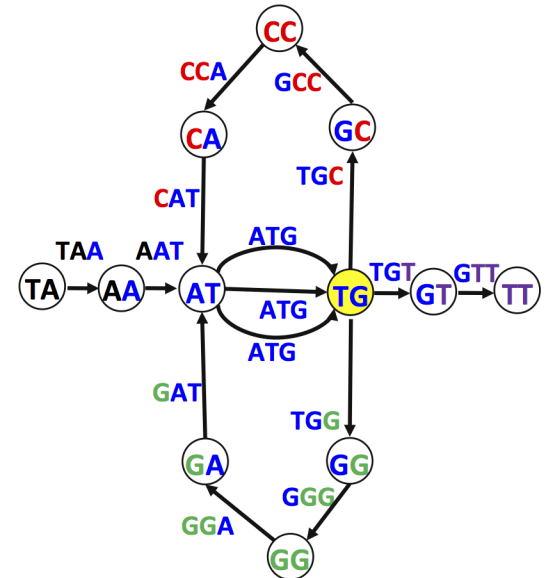
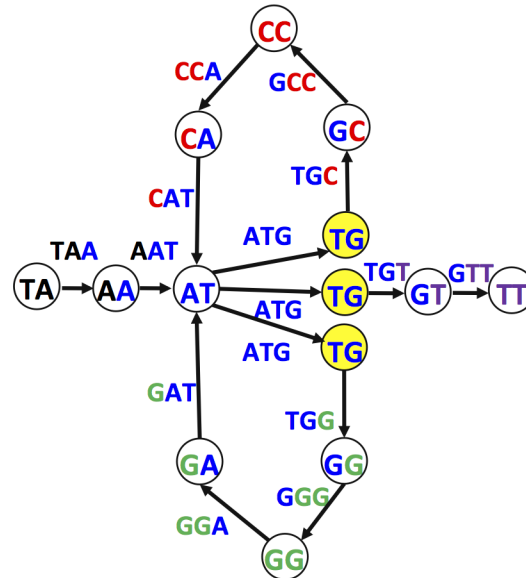
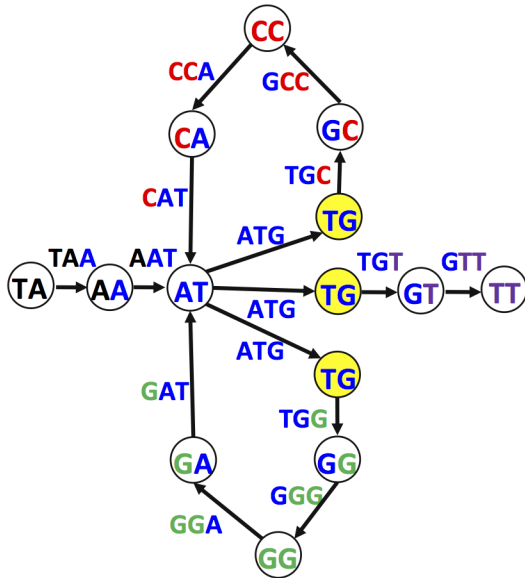
De Bruijn le da al pegamento

- La utilidad de los grafos de De Bruijn está en ‘pegar’ los nodos con el mismo prefijo:



Grafo de De Bruijn: grafo dirigido donde todos los nodos de igual nombre se fusionan sin perder sus enlaces

De Bruijn le da al pegamento



de Bruijn: pseudocódigo

```
deBruijn(kmers)
  grafo={}
  para cada kmer en kmers
    añadir sufijo(kmer)->[] a grafo
    añadir prefijo(kmer)->[] a grafo
  para cada kmer en kmers
    pk<-prefijo(kmer)
    añadir a la lista en grafo[pk] sufijo(kmer)
  devolver grafo
```

Ejercicio

- Implementar la función deBruijn:
 - **entrada:**
 - `kmers` (secuencias de igual longitud ordenadas alfabéticamente)
 - **salida:** grafo de deBruijn correspondiente, como diccionario de adyacencia
- Ejemplo:
 - `kmers`: todos los 3-mers en TAATGCCATGGGATGTT
 - `salida`: {'AA': ['AT'], 'GT': ['TT'], 'CC': ['CA'], 'CA': ['AT'], 'GG': ['GA', 'GG'], 'GC': ['CC'], 'AT': ['TG', 'TG', 'TG'], 'GA': ['AT'], 'TG': ['GC', 'GG', 'GT'], 'TA': ['AA']}

Ciclo Euleriano*

- Ruta que visita **todas las aristas** en un grafo dirigido exactamente una vez
- Encontrar un camino euleriano en un grafo de De Bruijn = reconstruir una cadena a partir de un conjunto de fragmentos

* Se pronuncia 'oileriano'

Euler vs Hamilton

¿Por qué molestarnos con Euler/De Bruijn si el problema ya lo podíamos solucionar con Hamilton?

Ensamblado

- El problema del ensamblado
- Grafos de De Bruijn
- **Teorema de Euler**
- Consideraciones adicionales

Teorema de Euler

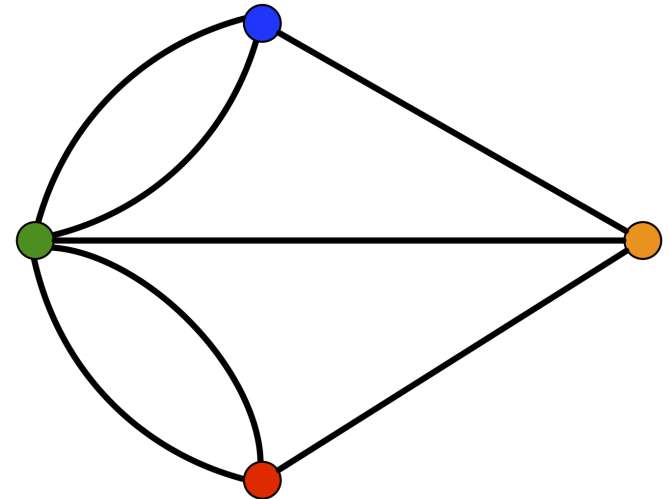
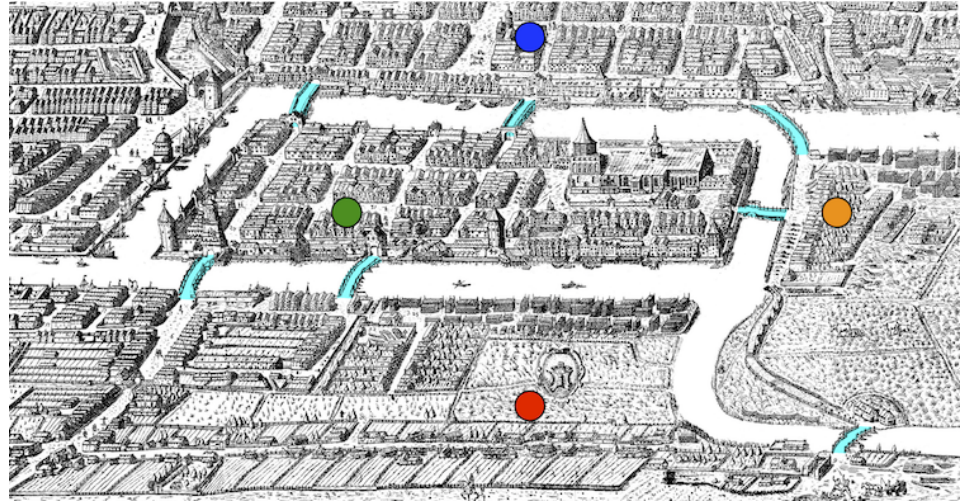
Los 7 puentes de Königsberg

El teorema de Euler

Grafos balanceados y ciclos eulerianos

Los 7 puentes de Königsberg

- ¿Podemos dar un paseo por Königsberg pasando una sola vez por cada uno de sus puentes?
- Pregunta equivalente:
 - ¿Hay algún ciclo Euleriano en el grafo correspondiente?
- Euler demostró que *no* era posible

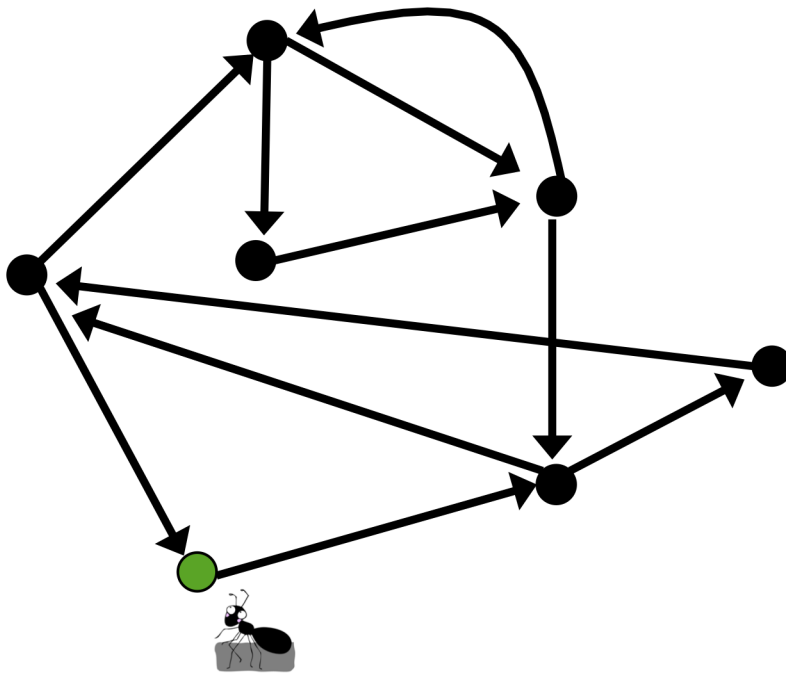


Grafo balanceado

- **Grado de salida/entrada** de un nodo v
 - Número de aristas que salen/entran al nodo
 - Los llamaremos $in(v)$ y $out(v)$
- Un grafo está **balanceado** si
 - $in(v)=out(v)$ para todo nodo v del grafo
- Un grafo **está fuertemente conectado** si
 - Existe una ruta entre dos nodos cualquiera del mismo

Teorema de Euler

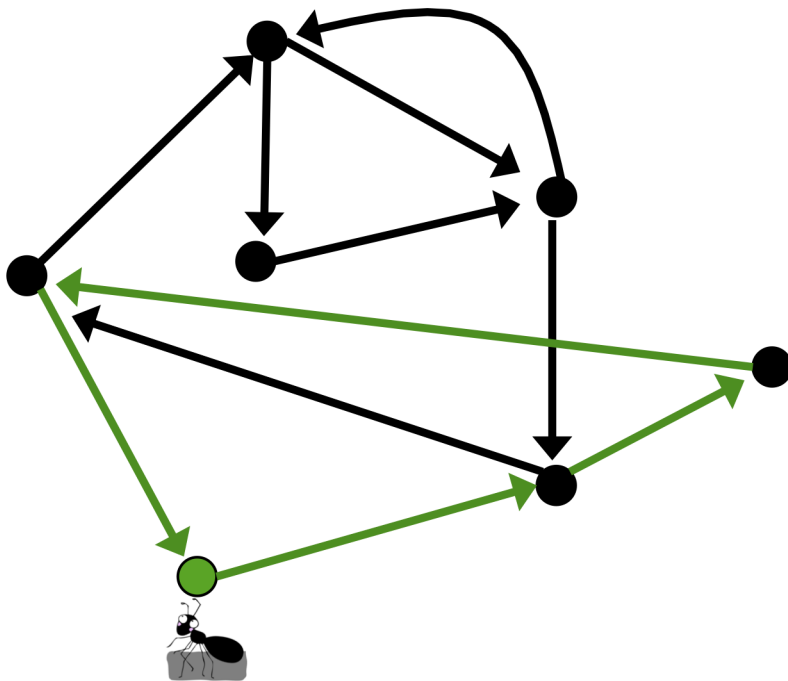
- Si un grafo g es euleriano (i.e. se puede trazar un ciclo euleriano) $\rightarrow g$ es balanceado*



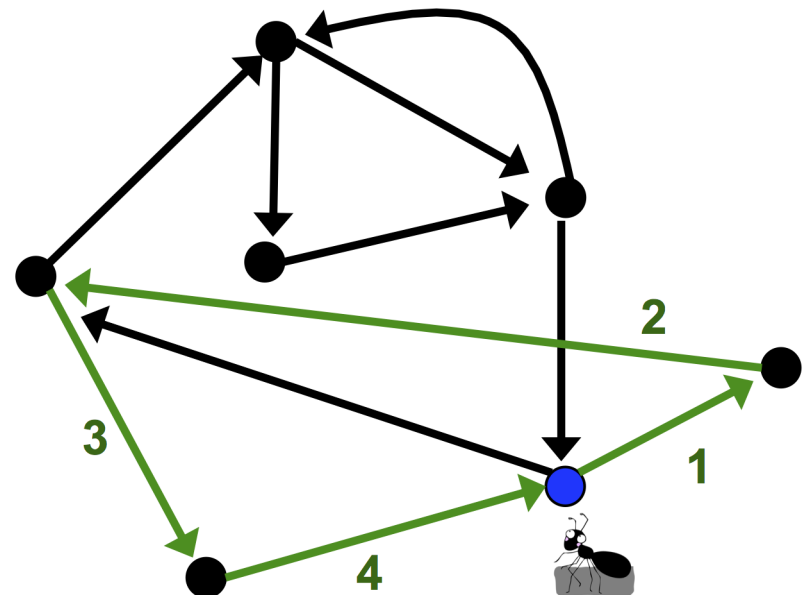
Sea una hormiga que se encuentra en un nodo v_0 (en verde)
Si empieza a caminar aleatoriamente por aristas sin repetir ninguna en este grafo balanceado y conectado ¿dónde se puede 'atascar'?

Demostración: Euler y la hormiga

- Sólo se puede atascar en el nodo de inicio

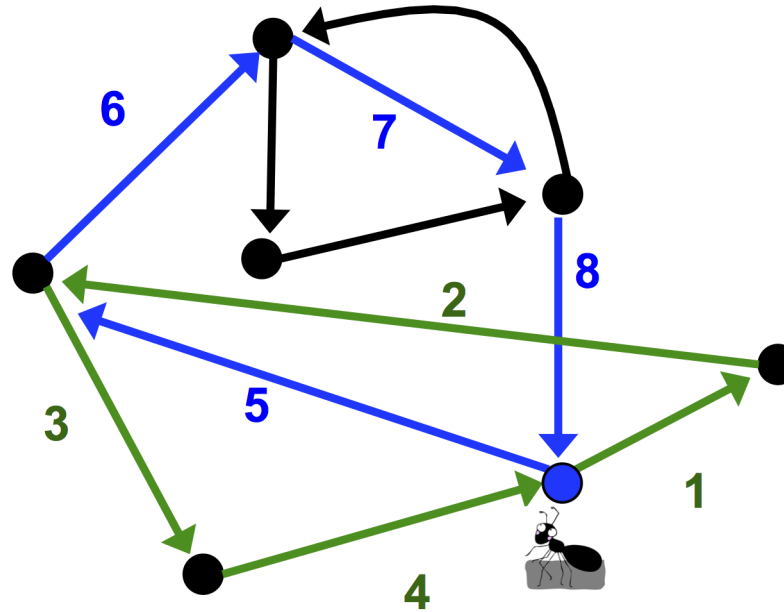


La hormiga, en un camino aleatorio, forma el $ciclo_0$ (verde)



Luego la obligamos a comenzar de nuevo en otro nodo de $ciclo_0$ con aristas sin recorrer, haciéndola repetir $ciclo_0$

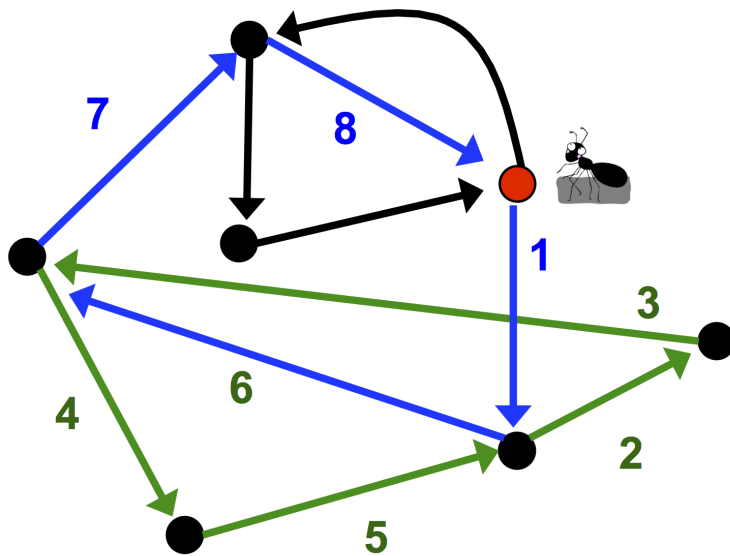
Euler y la hormiga



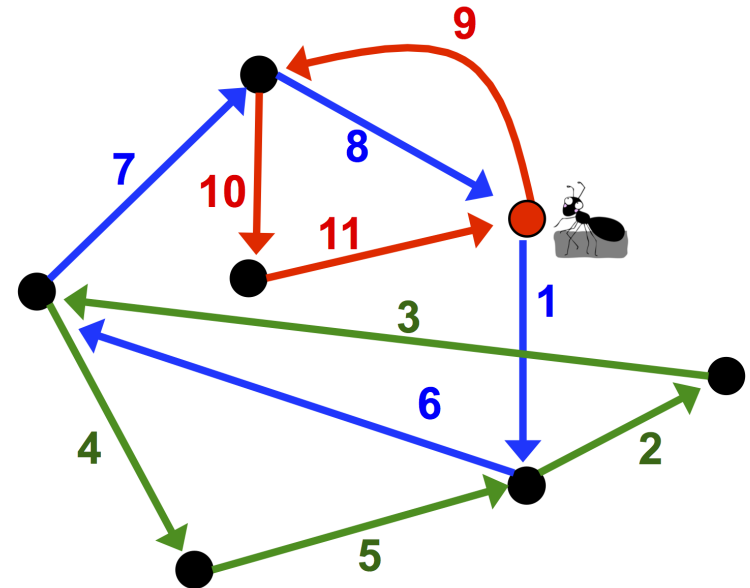
Continuará tras $ciclo_0$ recorriendo aristas aleatoriamente, hasta quedar atascada en un nuevo $ciclo_1$ (verde + azul)

Si este nuevo ciclo es euleriano, hemos terminado, si no, tendremos que buscar un nuevo nodo de inicio con aristas sin recorrer

Euler y la hormiga



Comenzamos ahora en un nuevo nodo (en rojo) con aristas sin recorrer en $ciclo_1$ y recorreremos $ciclo_1$



Al llegar de nuevo al nodo rojo tenemos aún aristas que recorrer. Continuamos hasta atascarnos y esta vez sí, $ciclo_2$ es euleriano

Ciclo euleriano: pseudocódigo

```
eulerianCycle(grafo)
  ciclo <- camino aleatorio en grafo sin
    visitar dos veces el mismo nodo
  mientras haya nodos sin explorar en grafo
    comienzo <- nodo en ciclo con aristas sin
      explorar
    ciclo' <- ruta que atraviese ciclo desde
      comienzo y luego continúe aleatoriamente
    ciclo <- ciclo'
  devolver ciclo
```

Euler vs Hamilton

- La búsqueda de:
 - caminos hamiltonianos es NP completa
 - ciclos eulerianos es factible computacionalmente
 - Siempre que el grafo esté balanceado

Ejercicio

- Implementar la función `balanced`:
 - **entrada:**
 - grafo (como un diccionario de adyacencia)
 - **salida:** `True` si el grafo está balanceado, `False` si no
- Ejemplo:
 - grafo: `{'AA': ['AT'], 'GT': ['TT'], 'CC': ['CA'], 'CA': ['AT'], 'GG': ['GA', 'GG'], 'GC': ['CC'], 'AT': ['TG', 'TG', 'TG'], 'GA': ['AT'], 'TG': ['GC', 'GG', 'GT'], 'TT': [], 'TA': ['AA']}`
 - salida: `False`

Ejercicio

- Implementar la función `eulerianCycle`:
 - **entrada:**
 - grafo (representado mediante un diccionario de adyacencia como el resultante de `deBruijn`)
 - NOTA: puede ser necesario convertirlo a ciclo añadiendo una arista entre el último nodo y el primero
 - **salida:** ciclo euleriano en grafo
- Ejemplo:
 - entrada: grafo de deBruijn realizado con todos los 3-mers en TAATGCCATGGGATGTT
 - Es decir: `deBruijn(composition('TAATGCCATGGGATGTT', 3))`
 - salida: ['AA', 'AT', 'TG', 'GG', 'GG', 'GA', 'AT', 'TG', 'GC', 'CC', 'CA', 'AT', 'TG', 'GT', 'TT', 'TA', 'AA']
 - Nota: puede salir una ruta distinta, pero siempre debe recorrer todas las aristas en un ciclo en el grafo de la diap. 17

Ejercicio 1

10 puntos

- Implementar la función `stringReconstruction`:
 - **entrada:**
 - `kmers` (conjunto de fragmentos del mismo tamaño)
 - **salida:** cadena reconstruida a partir de los fragmentos
- Ejemplo:
 - `kmers: composition( 'TAATGCCATGGGATGTT' , 3 )`
 - `salida: 'TAATGCCATGGGATGTT'`
 - Nota: puede salir una ruta alternativa, por ejemplo `'TAATGGGATGCCATGTT'` pero siempre debe recorrer todas las aristas en un ciclo en el grafo de la diap. 17
- Prueba: Utilizar los 10-mers en este enlace
 - <http://vis.usal.es/rodrigo/documentos/bioinfo/avanzada/datos/10mers.txt>
 - La salida debe comenzar por el prefijo que no tiene aristas de entrada.
 - En este caso, no hay rutas alternativas.

Ensamblado

- El problema del ensamblado
- Grafos de De Bruijn
- Teorema de Euler
- **Consideraciones adicionales**

Consideraciones adicionales

Longitud de los fragmentos

Pares de lectura

Cobertura, contigs y errores

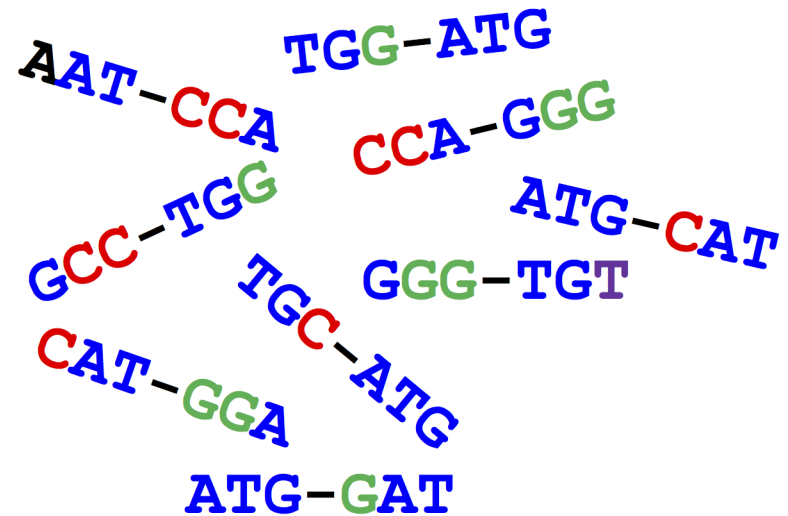
Reconstrucción y longitud de fragmentos

- Incrementar la longitud de los fragmentos ayuda en la reconstrucción de la cadena
 - Repite el ejemplo anterior con $k=4$ y $k=5$ en TAATGCCATGGGATGTT
- No hay una técnica que permita aumentar indefinidamente la longitud de las lecturas
 - Los secuenciadores modernos dan longitudes de unas 300 bps como máximo**

** Aunque hay nuevas tecnologías que ya pueden llegar a 1Kbps

Read pairs

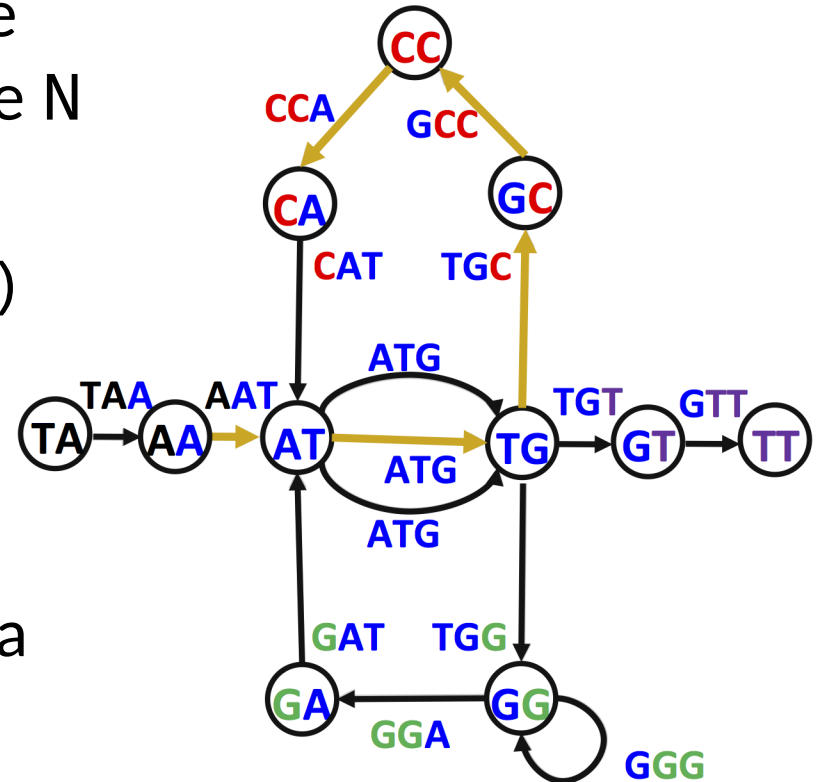
- Pares de lectura separados una distancia d en el genoma
 - Equivalente a una lectura larga con un 'salto', de longitud $k+d+k$, donde el segmento intermedio de longitud d es desconocido
 - Denominados (k,d) -mers
- Técnicamente factibles
 - Y si pudiéramos inferir la secuencia en el hueco, incrementaríamos la longitud de nuestras lecturas de k a $2k+d$



Pares de lectura de longitud 3 con un salto de longitud 1, es decir $(3,1)$ -mers

Read pairs

- Sea $\mathcal{L}$ lecturas una colección de todos los $2N$ k -mers obtenidos de N read pairs
- Sea $g = \text{deBruijn}(\mathcal{L})$
- Un read pair corresponderá a una ruta de longitud $k+d+1$ en g



La ruta resaltada de longitud $k+d+1=3+1+1=5$ entre AAT y CCA es AATGCCA, recuperando el hueco en AAT-CCA

Composición de pares

- La recuperación de los huecos sólo es posible si hay una ruta **única** para el read pair
 - Esto no es siempre ocurre
- Podemos elaborar un grafo de De Bruijn a partir de (k,d) -mers (grafo de pares)
 - Estos grafos **no garantizan** que cualquier ciclo euleriano en ellos sea una cadena válida

Complicaciones: cobertura

- Cobertura perfecta: **todos** los k-mers del genoma están secuenciados
 - Normalmente, este no es el caso
 - Solución: ‘romper’ los k-mers en secuencias de menor tamaño
 - k-mers muy pequeños: grafo complicado
 - k-mers muy grandes: cobertura no perfecta
- } *equilibrio*

Complicaciones: cobertura (ii)

ATGCCGTAGGACAACGACT
ATGCCGTAGG
GCCGTAGGAC
GTAGGACAAC
GACAACGACT
1122233334332221111

*Cuatro 10-mers sin cobertura perfecta (izquierda)
Si se trocean en 5-mers, dan cobertura perfecta (derecha)*

ATGCCGTAGGACAACGACT
ATGCC
TGCCG
GCCGT
CCGTA
CGTAG
GTAGG
TAGGA
AGGAC
GGACA
GACAA
ACAAC
CAACG
AACGA
ACGAC
CGACT
1234555555555554321

Complicaciones: contigs

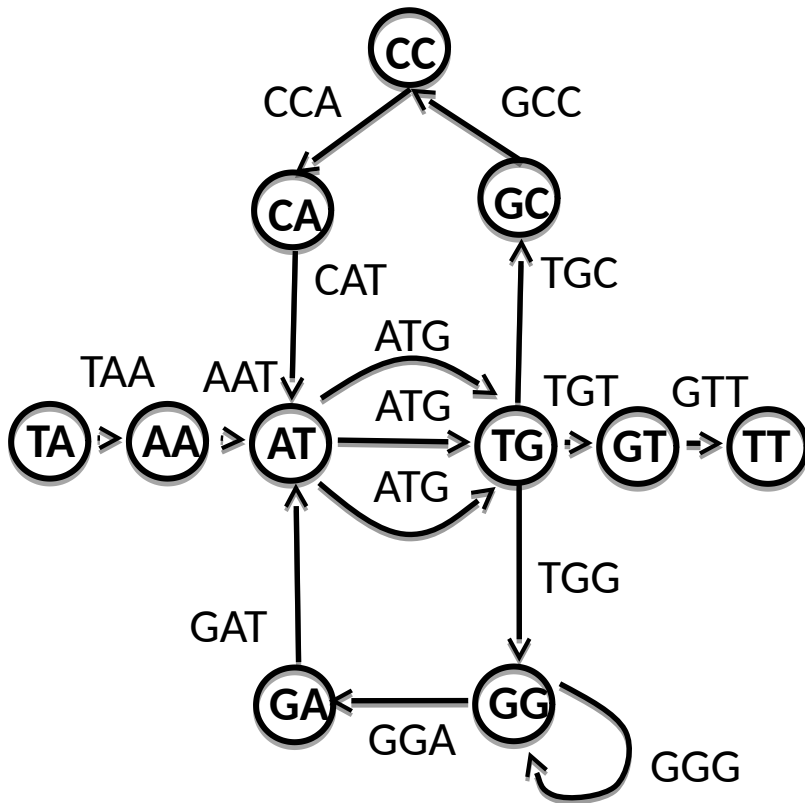
- Incluso tras romper las lecturas en k-mers pequeños, no se suele lograr cobertura total
 - Consecuencia: se ensamblan **contigs**, segmentos largos contiguos del genoma

La secuenciación típica de una bacteria resulta en unos 100 contigs, cada uno de entre 1.000 y 100.000 nucleótidos

- **Camino sin ramas:** $\text{in}(v)=\text{out}(v)=1$ para todo nodo v en el camino
 - Cada camino sin ramas de longitud máxima es un contig en un grafo de De Bruijn

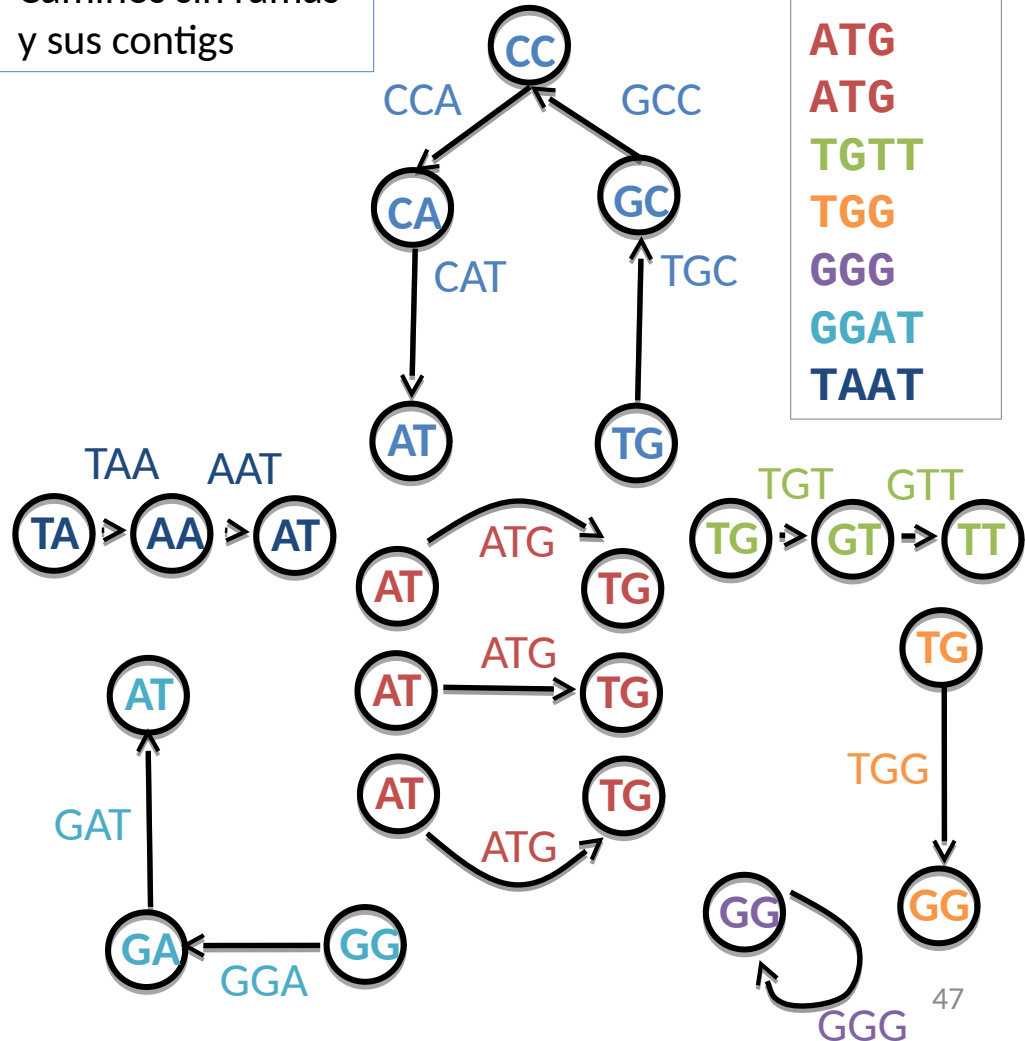
Complicaciones: contigs (ii)

Secuencia original y grafo de deBruijn



TAATGCCATGGGATGTGTT

Caminos sin ramas y sus contigs



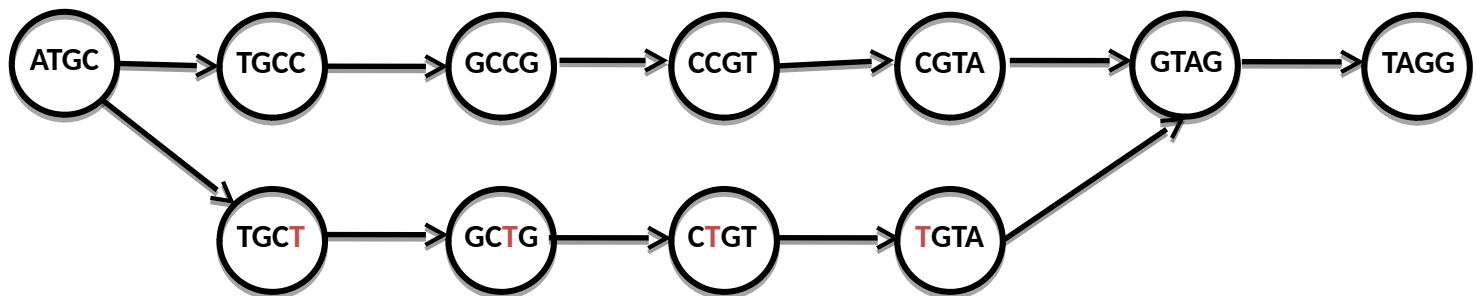
Ejercicio 2

10 puntos

- Implementar la función `contigs`:
 - **entrada:**
 - `kmers` (conjunto de fragmentos del mismo tamaño)
 - **salida:** todos los contigs en `deBruijn(kmers)`
- Ejemplo:
 - `kmers: composition('TAATGCCATGGGATGTT', 3)`
 - `salida: ['TAAT', 'ATG', 'ATG', 'ATG', 'TGTT', 'TGG', 'GGG', 'GGAT', 'TGCCAT']`
- Prueba: Utilizar los 10-mers en este enlace
 - <http://vis.usal.es/rodrigo/documentos/bioinfo/avanzada/datos/10mers.txt>
 - ¿Cuántos contigs se obtienen? ¿De qué longitudes?

Complicaciones: errores

- Las lecturas pueden contener errores
- Por ejemplo un error en el 10-mer de la diapositiva 45: ATGCTGTAGG
 - Da lugar a 5 errores al romperlo en 5 mers
 - ATCGT, TCGTC, GCTGT, CTGTA, TGTAG
 - Que generan una ruta errónea de ATGC a GTAGG



Complicaciones: errores

- Estas rutas erróneas se llaman **burbujas**
 - dos rutas disjuntas cortas que conectan los mismos nodos inicial y final
- Los ensambladores tratan de eliminarlas
 - Pero hay millones de ellas en un genoma
 - Y a veces se elimina la ruta correcta en vez de la incorrecta, añadiendo errores en vez de arreglarlos

Ejercicio final*

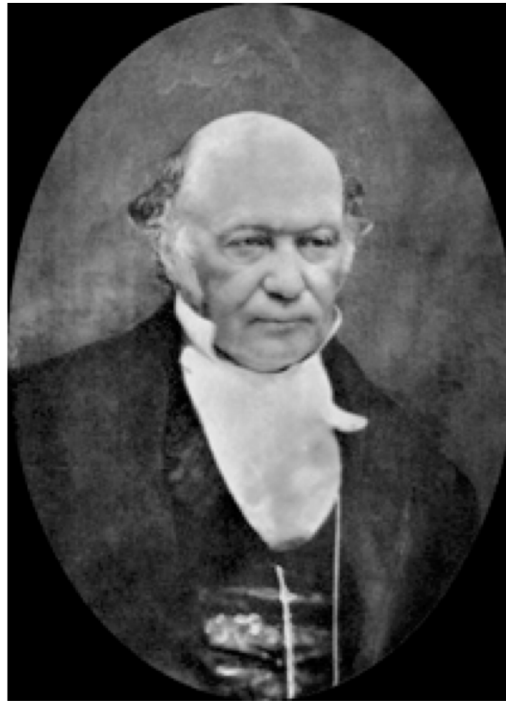
- Ensamblar el genoma de *Carsonella ruddii*
 - http://www.ncbi.nlm.nih.gov/nuccore/NC_018414.1
 - El enlace FASTA contiene el genoma ensamblado:
 - Dividir en secuencias de 70 bases (serán nuestros kmer s)
 - Re-ensamblar con los programas desarrollados
 - ¿Qué dificultades encuentras? ¿Cuántos contigs obtienes? ¿Cuánto se diferencia del genoma ensamblado en NCBI?

Carsonella ruddii es una bacteria que vive simbióticamente en el interior de algunos insectos.

Esta vida 'bajo tejado' le ha servido para reducir su genoma a sólo unas 160 000 bases. De hecho, los biólogos conjeturan que quizás esté perdiendo su identidad como bacteria y se termine convirtiendo en un orgánulo más del genoma del insecto. Esto ha ocurrido muchas veces durante la evolución. El ejemplo más conocido es el de la *mitocondria*, responsable de la producción de energía en las células humanas. Hace mucho tiempo, era una bacteria que triscaba libremente por los montes.



Leonhard Euler



William Hamilton



Nicolaas de Bruijn

Los matemáticos te preguntan.... ¿Cuál fue el titular del NYT del 27 de junio de 2000?